Navigating DataFrames and Series

Section 1: Learn

What are DataFrames and Series in Pandas?

Pandas provides two main data structures for handling data efficiently:

1. **Series** – A one-dimensional labeled array (like a column in Excel).
2. **DataFrame** – A two-dimensional table with labeled rows and columns (like a spreadsheet).

Why Use DataFrames and Series?

- **Efficient Data Handling** – Easily store, access, and manipulate large datasets.
- **Better than Lists & Dictionaries** – Offers powerful functions for filtering, aggregating, and analyzing data.
- **Compatible with Other Libraries** – Works with NumPy, Matplotlib, and Scikit-Learn.

How to Work with DataFrames and Series?

1. **Creating Series and DataFrames** – Using dictionaries, lists, and NumPy arrays.
2. **Selecting and Filtering Data** – Accessing specific rows, columns, or subsets.
3. **Applying Functions** – Performing calculations and transformations.

Real-Life Applications of DataFrames and Series

- **Finance** – Stock market analysis.
- **Healthcare** – Patient data processing.

- **E-commerce** – Customer transaction tracking.

Anecdote: How DataFrames Helped in COVID-19 Data Analysis

During the pandemic, researchers used **Pandas DataFrames** to track cases, recoveries, and deaths worldwide.

By analyzing large datasets efficiently, they identified **infection trends and helped policymakers take timely actions**.

Section 2: Practice

1. Creating a Pandas Series

```
import pandas as pd


# Creating a Series from a list
marks = pd.Series([85, 90, 78, 92, 88], index=['Amit', 'Priya', 'Rahul', 'Neha',
'Ravi'])
print(marks)


# Accessing elements in a Series
print(marks['Priya'])  # Output: 90
```

---

2. Creating a DataFrame

```
# Creating a DataFrame from a dictionary
data = {
    "Name": ["Amit", "Priya", "Rahul", "Neha"],
    "Age": [21, 22, 20, 23],
```

```
    "Marks": [85, 90, 78, 92]
}

df = pd.DataFrame(data)
print(df)
```

---

3. Accessing and Filtering Data

```
# Selecting a single column
print(df["Marks"])

# Selecting multiple columns
print(df[["Name", "Marks"]])

# Filtering rows where Marks > 80
high_scorers = df[df["Marks"] > 80]
print(high_scorers)
```

---

4. Common DataFrame Operations

```
# Adding a new column
df["Pass"] = df["Marks"] > 40
print(df)

# Sorting Data by Marks
df_sorted = df.sort_values(by="Marks", ascending=False)
```

```
print(df_sorted)


# Getting basic statistics
print(df.describe())
```

Section 3: Know More

Frequently Asked Questions (FAQs)

1. What is the difference between a Series and a DataFrame?

- **Series** → 1D, similar to a column.
- **DataFrame** → 2D, multiple rows and columns.

2. How do I read data from an Excel or CSV file?

Use pd.read_csv("file.csv") or pd.read_excel("file.xlsx").

3. How do I check for missing values?

```
df.isnull().sum()
```

4. How do I reset the index in a DataFrame?

```
df.reset_index(drop=True, inplace=True)
```

5. Why is Pandas important for data analysis?

- **Handles large datasets easily.**
- **Provides powerful filtering and transformation tools.**
- **Works well with visualization and machine learning libraries.**

Conclusion:

Pandas **Series and DataFrames** make data analysis **faster and more efficient**. They provide tools for **data manipulation, filtering, and visualization**, making them essential for **data science and analytics**.